

Coroutines belong in a TS

Document Number:	P0158R0
Date:	2015-11-06
Revises:	None
Project:	Programming Language C++
Project number:	TS 18822
Audience:	EWG
Reply-to:	Jamie Allsop (jamie.allsop@gmail.com) Jonathan Wakely (cxx@kayari.org) Christopher Kohlhoff (chris@kohlhoff.com) Anthony Williams (anthony@justsoftwaresolutions.co.uk) Roger Orr (rogero@howzatt.demon.co.uk) Andy Sawyer (andy.sawyer@gmail.com) Jonathan Coe (jbcoc@me.com) Arash Partow (arash@partow.net)

Abstract

This is a position paper by a number of concerned authors who share a strong feeling that any coroutines proposal belongs in a TS and should not be rushed into C++17. To be clear we are keen to see some sort of coroutine support in C++ but only after sufficient experience is obtained. Furthermore we feel there are unaddressed risks associated with the proposal currently being considered.

As such we believe the feature would be better evolved into, and through, the Technical Specification vehicle, allowing it to be refined and tempered through experience, rather than being hurried into C++17 for later regrets and surprises. This is much more of a radical change than some things we're putting through TSs.

Table of Contents

- [1. Introduction](#)
- [2. Motivation and Concerns](#)
 - [2.1 Suspend Up](#)
 - [2.2 Suspend Down](#)
 - [2.3 Why Suspend Down Matters](#)
 - [2.4 Significant Experience with Suspend-Down](#)
 - [2.5 Large Body of Experience and Important Use Cases Ignored](#)
 - [2.6 Limited Exposure to High Performance Domains](#)
 - [2.7 A "make-the-syntax-work" Approach](#)
 - [2.8 Risks Revisited](#)
 - [2.8.1 Maintenance Risks](#)
 - [2.8.2 Security and Performance Risks](#)
 - [2.8.3 Correctness Risks](#)
 - [2.8.4 Lock-in Risks](#)
 - [2.9 Summary](#)
- [3. Preferred Direction](#)

- [4. Conclusions](#)
- [Acknowledgements](#)
- [Appendix 1: Coroutine Papers](#)
- [Appendix 2: Related Papers](#)

1. Introduction

There have been a number of proposals that seek to bring "coroutines" to C++, under a [variety of names](#) but we will collectively refer to them as "coroutines" throughout this paper. There have also been several implementations in a variety of forms, targeting both language and library. [Appendix 1](#) enumerates the papers. This is a good thing and has provided all interested parties with plenty to discuss. Additionally the debates have, to some degree, led to improvements in all the proposals.

However recently there has been interest expressed in selecting one of the proposals (whose most recent incarnation can be found in [P0057R0 - Wording for Coroutines \(Revision 3\)](#)) for inclusion in C++17. The authors of this position paper feel that would be a mistake, and one not easily undone. Instead we believe that, should there be sufficient support for the proposal, then it belongs in a Technical Specification where it can enjoy more bake time and wider use.

Furthermore we believe there are significant short-comings in, and risks associated with, the proposal and would welcome an alternative coroutines TS that addresses those concerns so the community can better appreciate the options available.

2. Motivation and Concerns

Coroutines manifest themselves in languages for many different use-cases and concerns, making this a feature with an extremely broad and extensive impact. At a micro-level it is, without much effort, straight-forward to reason about the syntax and expected behaviour of coroutine facilities in a variety of example contexts. However at a macro level only determined usage experience can shed light on the issues that become important over time. Some of these issues can have a large negative impact and we feel it is critically important that those issues are treated with respect.

Some of the authors of this paper do have that more extensive usage experience of coroutine facilities, both in C++ and other languages such as Python, and therefore are aware of some of those issues. It is that real-world experience coupled with genuine technical concerns over the direction of [P0057R0](#) that have prompted us to strongly object to rushing the proposal into C++17.

To be clear, we have already identified a number of **risks** based on both user experience and technical examination of the [P0057R0](#) proposal. We will [revisit these later](#) but to summarise these include:

- severe maintenance risks,
- security risks,
- correctness risks.

Throughout the debates on coroutines we have seen numerous mis-understandings and wild claims making it difficult to have a well anchored debate. Part of this has been unavoidable due to the general diversity of experience and the looseness of terms. Attempts have been made throughout the discussions to clarify the terms used, their origin and their meaning (by all involved) but it is fair to say that there are still misconceptions and differences.

Further, critical debates about coroutines have been marred by mis-understandings regarding fundamental issues, or particular aspects of each proposal. For example minutes of one discussion discounted one proposal as not fit for purpose because it made use of type-erasure. In fact the proposal did not rely on type-erasure but the preferred proposal did!

That's just one example, but we have in general enjoyed a lack of clarity over the implications of various terms. For example,

- *stackless* and *stackful* coroutines, how are they defined?
- are stackful coroutines implicitly slower than stackless coroutines?
- what stack or stacks are we talking about?
- does *stacks* actually refer to stacks or is it being used to indicate a calling convention?
- is language support required, or not?
- do we need additional keywords, or not?
- is the hard-coding of use cases for specific needs, like *await*, the right approach?

- do we even know the complete set of use-cases (like *await*) or should that set be open ended?
- if the set is open ended can it be extended?
- can a library solution be viable with minimal enabling compiler hooks?
- what optimisations can be achieved?
- what optimisations are *guaranteed*?
- and so on

This has in large part been a natural consequence of the evolving shared understanding within the committee, but nevertheless it does point to a learning curve which we are all still on. One thing that has finally become clear is that there are at present two fundamentally different approaches to coroutine support, and that the answers to most of the questions asked above naturally fall out of the solutions to each. Those fundamentally different approaches are based on what **suspension model** is adopted. The concept of a suspension model was first introduced in:

- [N4232 - Stackful Coroutines and Stackless Resumable Functions](#)

and then discussed further in:

- [P0099R0 - A low-level API for stackful context switching](#),
- [P0073R0 - On unifying the coroutines and resumable functions proposals](#) and
- [P0114R0 - Resumable Expressions](#).

To summarise there are two suspension models, **suspend-up** (also referred to as *suspend up-and-out* and *suspend-by-return*) and **suspend-down** (also referred to as *suspend-by-call*).

NOTE: This paper does not seek to offer an authoritative discussion of suspension models or their mechanics. Instead you should refer to one of the above papers. In particular [P0073R0 - On unifying the coroutines and resumable functions proposals](#) is worth reading as it decomposes the concept even further. The paper argues that we do not necessarily need to consider only two basic models and should instead reason about the underlying mechanics of the suspension - using those as building blocks on which to build higher level abstractions. However for the purpose of this discussion it useful to identify the suspension model as the key differentiator for existing proposals as they tend to fall into one of the two camps. Further, most other aspects of the various proposals are subject to QoI, but you can't, for example, optimise a keyword away.

2.1 Suspend-Up

Conceptually the *suspend-up* model works by the caller suspending its own execution before invoking a callee. Only after the callee returns, and is in its terminal state, will the caller continue. This model works by the callee delegating responsibility for suspension upwards to the caller, and is by definition "viral" in a code-base. The caller contains the suspension and in order to allow suspension through multiple abstraction layers, it must be propagated manually through the call stack. This is not transparent and typically the code must be annotated through the use of a keyword, such as `await`.

NOTE: It should also be noted that the viral aspect does not just force code duplication and extensive refactoring. It also limits exploratory changes. For example, trying a different concurrency model that makes use of cooperative multitasking is not a simple endeavour - you need to re-write your code.

Similarly it poses problems when interacting with thirdparty libraries. For example, consider a situation were you have code that calls thirdparty code which then calls your code. That is:

```
local -> thirdparty -> local
```

If we assume that you do not have control over the thirdparty code and it can't be changed to add a suspend-up annotation, such as `await`, then you are going to face difficulties using the suspend-up model since it would require refactoring of the thirdparty code.

2.2 Suspend-Down

The *suspend-down* model works by delegating the mechanics of suspension to what looks like an ordinary function. In other words the "suspension point" is encapsulated by normal scope boundaries allowing the caller to interact safely with coroutines without needing special modifications.

NOTE: Considering the case of interacting with thirdparty code again, as we did for suspend-up, we have a much better situation. Both the original call site can suspend the calls to the thirdparty code and the penultimate bit in the call chain can also suspend because the thirdparty code in the middle is none-the-wiser about being suspended at the call site, or by the back-calls to other local code performing a suspend.

2.3 Why Suspend-Down Matters

The suspend-down model is important because it allows code to be refactored to make use of facilities such as coroutines, but even more importantly it allows natural composition and clean de-coupling. This is critical to building more complex systems. Suspend-down allows us to provide facilities as primitives upon which a rich vocabulary of capabilities can be built.

To re-iterate, suspend-down works within the existing function call model. Therefore, just as with `std::thread`, we can incorporate ordinary functions and, perhaps more importantly, existing functions into our coroutines. Consequently, suspend-down lets us build coroutine models as library abstractions. It does not limit us to a particular model, nor does it embed type requirements into the language.

2.4 Significant Experience with Suspend-Down

It is important to revisit the topic of **risk** at this point in the paper. There is significant experience in C++ with library-based suspend-down models as an approach to implementing coroutines (cf. [N3985 - A proposal to add coroutines to the C++ standard library \(Revision 1\)](#)). In such models it has been shown that the risks that we have identified are mitigated. The only issue with pure library-based approaches is that they often lack the required enablers for adequate compiler support to facilitate an optimal implementation. Given the non-invasive aspect of suspend-down coroutines we firmly believe a concerted effort should be made to explore what enablers are required in the language in order to allow for optimal implementations.

2.5 Large Body of Experience and Important Use Cases Ignored

Building on that experience with suspend-down models there is also significant experience with suspend-up macro based implementations in popular heavily used libraries such as [Boost.Asio](#) (or indeed the [standalone version](#) of the library) to solve well understood and important use-cases. That experience also reinforces the maintenance issues with the suspend-up model and highlights the need to further explore possible solutions that make use of a suspend-down approach.

There is a more important point though, and that is there is a large body of experience that exists in building systems that are ideally suited to capitalising on the advantages that coroutines offer. Of particular note are the high performance exchange platforms found in capital markets. Such systems are required to process huge volumes of data at deterministic *best* worst-case latencies. Typically observed in microseconds across network boundaries, with latencies themselves measured to nanosecond resolution. Internal to a system we care about latencies in 10s or 100s of nanoseconds. This is not simply driven by a desire to have "fast" systems. There is a large body of complex regulations surrounding this domain requiring both accurate measurement and consistent performance.

To help anchor an appreciation of the performance constraints required consider that we recently added SG14 "game development and low-latency" as a new study group. The systems we describe here are very much at the bleeding edge of performance in C++ and could require latencies easily an order of magnitude better than you might typically be concerned about even in the gaming world, or even in the "hard" real-time DSP world (on commodity hardware) where some of the authors also have experience. Moreover, failing systems in this domain are the subject of regulatory investigations and in some cases criminal prosecution.

In other words, we do not see this as a thought-experiment. This stuff matters to us and we have to care about how well any solution fits with the work that we do. If we say we have concerns about the current direction and state that rushing this feature into C++17 is probably too soon, then we would hope some weight is attached to those concerns.

2.6 Limited Exposure to High Performance Domains

In the last section we talked about the "ultra-low latency, high throughput" systems that are typically encountered in the capital markets space and stated that such systems make heavy use of techniques such as coroutines in order to meet their demands. Several of the authors of this paper have extensive experience in that space. One aspect of those systems is that, by far, the most common deployment platform is Linux. What this means is that, while it is nice to have an implementation of

one proposal for coroutines in Visual C++, it is of little value to exploring the proposal in one of the main use-cases for the feature. This is very unfortunate because C++ is used in the domain outlined precisely because it affords an opportunity to gain the best performance from the hardware typically used.

Even if we set aside the concerns we have over proposals that rely on a suspend-up model, such as [P0057R0](#), we cannot ignore that a large body of use-cases are simply not catered for by the existence of an implementation in Visual C++. At best we can only explore it in detail with toy examples and cannot make use of it in large existing code-bases. Of course the viral effect of suspend-up coroutines makes that hard regardless.

2.7 A "make-the-syntax-work" Approach

There is a general consensus among the authors of this paper that the suspend-up [P0057R0](#) proposal being considered for fast-tracking into C++17 is very much the result of copying syntax from other languages and "making-it-work". We recognise this is not an unreasonable place to start.

However this also has a number of problems which we believe are significant.

- The proposal appears to be copying from other languages but **not** learning from the mistakes they have made. The issues we identify in this paper are not unique to [P0057R0](#). They exist in other languages such as Python which adopt a similar approach. We should not be blindly copying those mistakes across into C++.
- To some degree users are being told what they need by compiler writers, rather than compiler writers finding an efficient way to provide what users want. In fact it is the cross-pollination of ideas that are important here. Compiler-writers need to help users better understand what they can get. Instead we see some concerns being too easily dismissed. This is not good for such an extensive and invasive feature.
- There is too much focus on micro-level use so the wider picture is being lost. It is not very hard to write (and make work) trivial examples and of course it is important to get things right at the micro-level. However, if this is done at the expense of the macro-level view then we are in clear danger from creating future issues in the language that could otherwise have been avoided. More time is needed to adequately understand macro-level concerns.
- There appears to be a post-hoc approach to fixing some of the issues that have surfaced throughout the lifetime of the proposal. For example the suggestion to use the `proposed [[nodiscard]]` feels very much like an afterthought. This is not to say that the approach suggested is not a good one. It may well be a very good fit. The observation is that we feel there are issues such as this one that are subject to *late discovery*. This suggests more time is required to gain deeper insight.
- Lastly many of the comparisons with other languages have been based on comparisons to dynamic languages. This is fine up to a point but it also brings serious concerns. It is far too easy to adopt a syntax modelled on such languages that is quite bluntly inappropriate for a language like C++. While we may be able to optimise away some uses of type-erasure without hard guarantees it strongly suggests that we need more experience with the feature in C++. Most likely we can do better and a model that (viral suspension aside) might work in a dynamic language is probably going to be a sub-optimal approach in C++.

2.8 Risks Revisited

This section is here primarily to summarise the risks we see with the [P0057R0](#) proposal. The expected take-away should be that people with solid experience in this domain see issues with the proposal and those issues lead to recognisable risks we would like to avoid. As such it should reinforce the argument that rushing through such a proposal into C++17 is both premature and foolish.

2.8.1 Maintenance Risks

We have already discussed the viral aspect of coroutine proposals that adopt a *suspend-up* suspension model. With small simple examples or code that is rarely refactored you may not fully appreciate the impact this can have. With larger code-bases and more complex domains, particularly those with high performance demands, the impact is much more obvious. In such domains intelligent refactoring is essential in streamlining code. One of the authors of this paper has first hand experience of using suspend-up coroutines to build a high performance system and that experience has made him adamant that the viral aspect of suspend-up coroutines is both expensive and detrimental to the long term value of any code that adopts the approach. To quote his experience here:

[...] this concern was raised quite some time ago and it is a very important one. [...] the source of this complaint that was raised in P0114 was mainly from me. Our company (clearpool.io) received a substantive research grant a couple of years ago to develop a real-time web delivery platform as part of an electronic fixed income trading platform - basically order entry and market data over the internet. We used Python to handle incoming requests and made extensive use of coroutines that were built on top of a suspend up-and-out model - that's how they work in Python. The code-base was good and worked well initially but now - two years on - it needs completely re-written because refactoring it has become harder and harder. The reason is the viral suspension model. Which incidentally requires pretty much two versions of any function that might be used with the model, per the comment that P0114 made with respect to the impact on the standard library.

The main reason why I am so interested in *any* proposal that adopts a suspend down model is that it allows me to write refactorable and more importantly, generic code. At first I did not think it was possible to do this while retaining performance but now I am convinced it is - and P0114 helps shine a big light on that. Given that we have a chance to be one of the first languages to get this right I am amazed we are in such a rush to standardise a non-optimal, I'd even say broken, approach.

In clearpool.io we are looking to improve our C++ code base and are keen to adopt anything that will offer the benefits of coroutines but without the hideous maintenance burden. If we are ok with a maintenance burden we could use a macro-based approach for some class of problems and at least we know exactly what we're getting with that (and we do). [...] we are in the business of low-latency, high performance C++ systems so I'd like think we have at least some inkling of what we'd like to see and what we don't want.

Clearly this is somewhat emotive but the general message is important. Real user experience shows this is a real issue.

2.8.2 Security and Performance Risks

The design of [P0057R0](#) increases the risk of **unfairness**, **jitter**, **starvation**, and **susceptibility to denial-of-service attacks**. These are all very serious problems, with possibly extreme consequences in the capital markets domain, previously identified as a primary consumer of this feature.

Consider this code:

```
auto process_messages(Connection& c)
{
    while (c.is_open())
    {
        Message m = await receive_message(c);
        process_message(m);
    }
}
```

The two new concepts that will be embedded in the language are the type requirements for an *Awaitable* (which is returned by `receive_message()`), and the keyword `await` (which requires an *Awaitable*).

Let's say `process_message()` takes a certain time to execute, and by the time we loop around again to `await` a new message, there is already one available. Since the `await` keyword is specified as:

```
( await-ready-expr
  ? await-resume-expr
  : (await-suspend-expr, suspend-resume-point, await-resume-expr))
```

this essentially means that if the *Awaitable* is already ready (`await_ready` returns `true`) the coroutine continues without suspending. And so on and so on for an *indefinite number of iterations*. This can at best result in high jitter; and at worst starvation. If we were using real threads here then we would eventually be saved by the OS scheduler once our timeslice ran out. However the point of coroutines is to map a larger number of event chains on to a smaller number of threads, and we have no OS scheduler to save us.

The response to this concern is the assertion that the onus is on the library developers (in this case the developer of `receive_message()` to write "correct" *Awaitables*. That is, given:

```

auto some_async_function()
{
    for (;;)
    {
        await A();
        await B();
        await C();
    }
}

```

sensible library developers (the authors of `A()` , `B()` , and `C()`) will ensure that `await_ready` returns `false` , if they know that's what their use case calls for. We believe this attitude is not acceptable, we would ascertain that it's the author of `some_async_function()` that cares about this behaviour.

This is especially so since the stated goal of `await` is that we should be able to compose Awaitables from disparate sources. Scheduling ought to be a cross cutting concern for the coroutine, so we'd much rather see it as a library interface, something like:

```

spawn(always_suspend, some_async_function);

```

or:

```

spawn(always_continue, some_async_function);

```

We believe the field experience represented by Visual C++ is not valid here, because the support for `std::future` always spawns a new thread:

```

template<class _Ty>
void await_suspend(future<_Ty>& _Fut,
    experimental::coroutine_handle<> _ResumeCb)
{
    // change to .then when future gets .then
    thread _WaitingThread([&_Fut, _ResumeCb]{
        _Fut.wait();
        _ResumeCb();
    });
    _WaitingThread.detach();
}

```

By spawning a new thread, the starvation issue may be hidden from users. For those familiar with SG1 jargon, it means that the coroutines as presented in [P0057R0](#) using `std::future` provide the strongest *concurrent forward progress guarantee*, rather than the *parallel progress guarantee* which more likely characterises the use cases for coroutines.

2.8.3 Correctness Risks

The proposed [P0057R0](#) makes it difficult, and error prone, to write correct (race-free) code where multiple coroutines must access shared data.

Consider the case where we have two or more coroutines that access some shared state. We may want to pin them to a particular thread (or thread pool of size 1), or use a serial executor, or something else.

In [P0057R0](#), the `await_suspend` function takes only a `coroutine_handle`. By default, the call to `coroutine_resume` occurs in the context of the Awaitable. This means that the coroutine may jump from one thread to another. In our opinion it is ok to permit this behaviour, but it's definitely not ok to make it the default.

The response to this concern was that you can specify an executor, but it turns out you need to do it on *every* `await` operation. So you may have to write something like:

```

auto some_async_function()
{
    for (;;)
    {

```



```

        await on_executor(my_ex, A());
        await on_executor(my_ex, B());
        await on_executor(my_ex, C());
    }
}

```

If function `A()`, in turn, accesses the shared data then we need to ensure that any `await` s performed by `A()` *also* use the executor. There may be a solution for all this, but it is not part of the current proposed wording, and that once again highlights the need for more bake time.

Again, this should be a cross cutting concern and we probably want a library interface more like:

```

spawn(my_ex, some_async_function);

```

Once again, we believe the field experience does not support the proposed design, quite apart from its apparent lack of stability.

It turns out that C# *does* address this issue (at least from our understanding). In C# you have the notion of a `SynchronizationContext` (very similar concept to executors), and a `SynchronizationContext.Current` property that represents the active thread-local context. When you do an `await`, the current `SynchronizationContext` is captured, and when resuming the coroutine the `coro_handler.resume()` is submitted as a lambda to the `SynchronizationContext`, using the context's `Post()` member function.

2.8.4 Lock-in Risks

These are obvious risks but ones we'd like to reinforce and make explicit. If we do fast-track coroutines into C++17, in whatever form, we feel we may be locked-in to the model it promotes. In the case of [P0057R0](#) we believe that model is a sub-optimal choice. For all intents and purposes, we worry it will significantly diminish the likelihood of work continuing on any alternative suspend-down based proposals. Not because we think people will lose interest or motivation, but because it will likely become more difficult to convince people of the value of using committee time to explore a feature "we already have".

The fear is we'll be potentially stuck with something that is not fit-for-purpose in important domains and yet have no real opportunity to drive adoption of an alternative. This is particularly bad with [P0057R0](#) because of its invasive impact on the language and likely future idioms that will be required to make intelligent use of the feature. Had the proposal been more focused on primitives for a library-based approach we might have had some wiggle room, but in its current form there would be no going back.

2.9 Summary

In our mind these concerns have not been adequately addressed. We do not doubt there is a large body of competency with many aspects of the problems encountered in the space, and the space is broad, but we would also stipulate that it is far from enough to warrant fast-tracking [P0057R0](#) into C++17.

3. Preferred Direction

Our preferred direction is three-fold.

First, as stated in the title of this paper, and throughout, we strongly believe that the only sensible course of action with regard to coroutines is to place them into a TS. Given that this paper was motivated by a move to put the "suspend-up" coroutines ([P0057R0](#)) proposal into C++ 17 this can be read as, "if there is strong interest within the committee for this proposal then **put it into a TS, not C++17**".

Second, we strongly believe that the "suspend-up" suspension model chosen by [P0057R0](#) is a poor one and echoes the mistakes made in other languages. Additionally some of the side-effects brought by that choice, such as type-erasure, are less than ideal and bring with them their own issues (but we recognise those side issues may have good solutions over time). We would therefore strongly encourage work to be continued on an alternative "down" suspension model approach so that the committee has the opportunity to compare and evaluate the relative benefits and costs of both. It has been clear that general committee experience is lacking. In that light we therefore hope to have a separate TS capturing that approach.

Given the fundamental difference in the two approaches and the potential impact on the language it would be forward thinking to name a TS based on [P0057R0](#) with that in mind. For example, "Suspend-Up Coroutines" or an alternative, better name. This would open the door to "Suspend-Down Coroutines" or similar.

Third, any coroutines proposal needs some degree of compiler support. [P0057R0](#) has benefited significantly from having a compiler on which to experiment and implement. The generally fluid nature of, and many changes in, the proposal over its lifetime is a testament to that benefit. Given our desire to see work continue on the "down" suspension model approach we would strongly encourage any compiler vendor with an interest in this feature to engage with any authors of proposals in the area. As noted in the previous section GCC and Clang would be attractive compilers to work in. This would open the door to gaining real production experience in high performance systems that rely on Linux as their underlying operating system. As previously stated, several of the authors of this paper work in the ultra-low latency financial markets space and are willing to utilise any such proposal in their code-base to test for suitability.

4. Conclusions

There are two big red flags that people have been ignoring in the pursuit of adding some "big ticket" and "cool" feature to C++17.

The members of the committee who appear to have the most experience of utilising coroutines to solve real-world high performance problems seem to have little influence (for whatever reason) but these are voices that we should be listening carefully to. That there is vocal hesitancy by members with expertise in the area to adding coroutines to C++17 is one red flag.

The second red flag is that it is becoming clear that we have a number of TSes like the Parallelism, Concurrency, Networking and potentially two Coroutine TSes that are all screaming out for a more unified and consistent model, or at least well thought out interacting models.

We strongly feel that the benefit of a TS is that it buys us time to explore those relationships. It is probably a little too early to see all the benefits but we believe that in general there is positive movement forward, and the TS vehicle is giving us that opportunity. That we would select coroutines, with its potentially huge impact yet without any substantive user experience, and fast-track it into C++17, is in itself: a very bad idea.

Acknowledgements

The authors would like to thank Francis Glassborow and Kevlin Henney for ideas, and Ville Voutilainen for feedback and suggestions.

Appendix 1: Coroutine Papers

Chronological, most recent first

WG21 Number	Title	Author	Document Date	Mailing Date	Revises	Group	Newer?
P0162R0	A response to "P0055R0: On Interactions Between Coroutines and Networking Library"	Christopher Kohlhoff	2015-11-06	2015-11		LEWG	
P0159R0	Response To: Resumable Expressions P0114R0	Gor Nishanov	2015-11-06	2015-11		EWG	
P0114R0	Resumable Expressions (revision	Christopher	2015-09-25	2015-09	N4453	SG1	

	1)	Kohlhoff					
P0099R0	A low-level API for stackful context switching	Oliver Kowalke, Nat Goodspeed	2015-09-27	2015-09	N4397	SG1	
P0073R0	On unifying the coroutines and resumable functions proposals	Torvald Riegel	2015-09-25	2015-09		EWG	
P0071R0	Coroutines: Keyword alternatives	Gor Nishanov	2015-09-12	2015-09		EWG	
P0070R0	Coroutines: Return Before Await	Gor Nishanov	2015-09-12	2015-09		EWG	
P0057R0	Wording for Coroutines (Revision 3)	Gor Nishanov	2015-09-26	2015-09	N4499	CWG, LEWG	
P0056R0	Soft Keywords	Gor Nishanov	2015-09-12	2015-09		EWG	
P0055R0	On Interactions Between Coroutines and Networking Library	Gor Nishanov	2015-09-12	2015-09		LEWG	
P0054R0	Coroutines: reports from the fields	Gor Nishanov	2015-09-12	2015-09		EWG	
N4499	Draft wording for Coroutines (Revision 2)	Gor Nishanov, Daveed Vandevoorde	2015-05-22	2015-05	N4403	EWG	P0057R0
N4453	Resumable Expressions	Christopher Kohlhoff	2015-04-12	2015-04		EWG	P0114R0
N4403	Draft Wording for Resumable Functions	Gor Nishanov	2015-04-10	<i>missing</i>		EWG	N4499
N4402	Resumable Functions (revision 4)	Gor Nishanov, Jim Radigan	2015-03-31	<i>missing</i>	N4286	EWG	
N4398	A unified syntax for stackless and stackful coroutines	Oliver Kowalke	2015-04-09	2015-04		SG1	
N4397	A low-level API for stackful coroutines	Oliver Kowalke	2015-04-09	2015-04	N3985	SG1	P0099R0
N4287	Threads, Fibers and Couroutines (slides deck)	Gor Nishanov	2014-11-18	2014-11		EWG	
N4286	Resumable Functions (revision 3)	Gor Nishanov, Jim Radigan	2014-11-18	2014-11	N4134	EWG	N4402
N4244	Resumable Lambdas: A language extension for generators and coroutines	Christopher Kohlhoff	2014-10-13	2014-10		SG1	

N4232	Stackful Coroutines and Stackless Resumable Functions	Nat Goodspeed	2014-10-13	2014-10		SG1	
N4134	Resumable Functions v.2	G. Nishanov, J. Radigan	2014-10-10	2014-10	N3977	SG1	N4286
N4024	Distinguishing coroutines and fibers	O. Kowalke, N. Goodspeed	2014-05-22	2014-05		SG1	
N3985	A proposal to add coroutines to the C++ standard library (Revision 1)	O. Kowalke, N. Goodspeed	2014-05-22	2014-05	N3708	SG1	N4397
N3977	Resumable Functions	Niklas Gustafsson	2014-05-22	2014-05	N3858	SG1	N4134
N3858	Resumable Functions	N. Gustafsson, D. Brewis, H. Sutter	2014-01-19	2014-01	N3722	SG1	
N3722	Resumable Functions	N. Gustafsson, D. Brewis, H. Sutter, S. Mithani	2013-08-30	2013-09	N3650	SG1	N3858
N3708	A proposal to add coroutines to the C++ standard library	O. Kowalke, N. Goodspeed	2013-03-04	2013-09		LWG	N3985
N3650	Resumable Functions	N. Gustafsson, D. Brewis, H. Sutter, S. Mithani	2013-05-02	2013-05	N3564	SG1	N3722
N3564	Resumable Functions	N. Gustafsson, D. Brewis, H. Sutter, S. Mithani	2013-03-15	2013-03	N3328	SG1	N3650
N3328	Resumable Functions	Niklas Gustafsson	2012-01-12	2012-01		EWG	N3564

Appendix 2: Related Papers

Chronological, most recent first

WG21 Number	Title	Author	Document Date	Mailing Date	Revises	Group	Newer?
P0068R0	Proposal of <code>[[unused]]</code> , <code>[[nodiscard]]</code> and <code>[[fallthrough]]</code> attributes	Andrew Tomazos	2015-09-03	2015-09		EWG	